

```
In [ ]: import marimo as mo
import math
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

```
In [ ]: #1. Data
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_set = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_set = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
test_loader = DataLoader(test_set, batch_size=64, shuffle=False)
```

```
In [ ]: #2. Model
class MnistCNN(nn.Module):
    def __init__(self):
        super(MnistCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(3136, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x))) # → 32×14×14
        x = self.pool(torch.relu(self.conv2(x))) # → 64×7×7
        x = x.view(x.size(0), -1) # → 3136
        x = torch.relu(self.fc1(x)) # → 128
        x = self.fc2(x) # → 10 logits
        return x
```

```
In [ ]: #3. Setup
device = "cuda" if torch.cuda.is_available() else "cpu"
model = MnistCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

#4. Training
for epoch in range(10):
    model.train()
    correct, total = 0, 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
```

```

        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        preds = outputs.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    print(f"Epoch {epoch+1:02d} | Train Acc: {100*correct/total:.2f}%")

```

```

Epoch 01 | Train Acc: 84.74%
Epoch 02 | Train Acc: 95.23%
Epoch 03 | Train Acc: 96.91%
Epoch 04 | Train Acc: 97.63%
Epoch 05 | Train Acc: 98.09%
Epoch 06 | Train Acc: 98.31%
Epoch 07 | Train Acc: 98.50%
Epoch 08 | Train Acc: 98.64%
Epoch 09 | Train Acc: 98.78%
Epoch 10 | Train Acc: 98.84%

```

```

In [ ]: def evaluation():
        #5. Evaluation
        model.eval()
        correct, total = 0, 0

        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                preds = outputs.argmax(dim=1)
                correct += (preds == labels).sum().item()
                total += labels.size(0)
        return print(f"\nFinal Test Accuracy: {100*correct/total:.2f}%")

evaluation()

```

Final Test Accuracy: 98.61%

Previous FCN Model

```

In [ ]: def FCN_primary():
        #2. Weight init (784 → 128 → 64 → 10)

        device = "cuda" if torch.cuda.is_available() else "cpu"

        def init_params():
            w1 = torch.randn(128, 784, device=device) * math.sqrt(2.0 / 784)
            b1 = torch.zeros(128, 1, device=device)
            w2 = torch.randn(64, 128, device=device) * math.sqrt(2.0 / 128)
            b2 = torch.zeros(64, 1, device=device)
            w3 = torch.randn(10, 64, device=device) * math.sqrt(2.0 / 64)
            b3 = torch.zeros(10, 1, device=device)

```

```
    return w1, b1, w2, b2, w3, b3
```

#3. Forward pass

```
def forward(w1, b1, w2, b2, w3, b3, X):
    z1 = torch.matmul(w1, X) + b1
    a1 = torch.relu(z1)
    z2 = torch.matmul(w2, a1) + b2
    a2 = torch.relu(z2)
    z3 = torch.matmul(w3, a2) + b3
    exp_z3 = torch.exp(z3 - z3.max(dim=0, keepdim=True)[0])
    a3 = exp_z3 / exp_z3.sum(dim=0, keepdim=True)
    return z1, a1, z2, a2, a3
```

#4. Loss(cross-entropy)

```
def compute_loss(a3, Y_onehot):
    return -(1 / Y_onehot.size(1)) * (Y_onehot * torch.log(a3 + 1e-8)).sum()
```

#5. Backward pass

```
def backward(z1, a1, z2, a2, a3, w2, w3, X, Y_onehot):
    m = Y_onehot.size(1)
    dz3 = a3 - Y_onehot
    dw3 = (1/m) * torch.matmul(dz3, a2.T)
    db3 = (1/m) * dz3.sum(dim=1, keepdim=True)
    dz2 = torch.matmul(w3.T, dz3) * (z2 > 0).float()
    dw2 = (1/m) * torch.matmul(dz2, a1.T)
    db2 = (1/m) * dz2.sum(dim=1, keepdim=True)
    dz1 = torch.matmul(w2.T, dz2) * (z1 > 0).float()
    dw1 = (1/m) * torch.matmul(dz1, X.T)
    db1 = (1/m) * dz1.sum(dim=1, keepdim=True)
    return dw1, db1, dw2, db2, dw3, db3
```

#6. Training

```
w1, b1, w2, b2, w3, b3 = init_params()
lr = 0.01
```

```
for epoch in range(15):
    correct, total = 0, 0
```

```
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        X = images.view(images.size(0), -1).T # (784, batch)
        Y = torch.zeros(10, labels.size(0), device=device)
        Y.scatter_(0, labels.unsqueeze(0), 1) # one-hot
```

```
        z1, a1, z2, a2, a3 = forward(w1, b1, w2, b2, w3, b3, X)
        dw1, db1, dw2, db2, dw3, db3 = backward(z1, a1, z2, a2, a3, w2, w3, X, Y)
```

```
        w1 -= lr * dw1; b1 -= lr * db1
        w2 -= lr * dw2; b2 -= lr * db2
        w3 -= lr * dw3; b3 -= lr * db3
```

```
        preds = a3.argmax(dim=0)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
```

```
print(f"Epoch {epoch+1:02d} | Train Acc: {100*correct/total:.2f}%")
```

```

#7. Evaluation
correct, total = 0, 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        X = images.view(images.size(0), -1).T
        _, _, _, a3 = forward(w1, b1, w2, b2, w3, b3, X)
        preds = a3.argmax(dim=0)
        correct += (preds == labels).sum().item()

        total += labels.size(0)
    return print(f"\nFinal Test Accuracy: {100*correct/total:.2f}%")

```

FCN_primary()

Epoch 01		Train Acc: 85.69%
Epoch 02		Train Acc: 93.20%
Epoch 03		Train Acc: 94.66%
Epoch 04		Train Acc: 95.42%
Epoch 05		Train Acc: 96.00%
Epoch 06		Train Acc: 96.50%
Epoch 07		Train Acc: 96.87%
Epoch 08		Train Acc: 97.20%
Epoch 09		Train Acc: 97.41%
Epoch 10		Train Acc: 97.63%
Epoch 11		Train Acc: 97.86%
Epoch 12		Train Acc: 97.97%
Epoch 13		Train Acc: 98.15%
Epoch 14		Train Acc: 98.24%
Epoch 15		Train Acc: 98.46%

Final Test Accuracy: 97.09%

Models comparison

for 10 epochs

With convolution:

The training accuracy = 98.84%

Test data accuracy = 98.63%

FNC (no convolution)

Training accuracy = 97.88%

Test accuracy = 96.75%

So we can see there is an improvement when convolution is used. But the steepness is very pronounced when we look at how many epochs needed to cross 98% (its 5 vs 12).

Convolution filters got 98% accuracy is just 5 epochs compared to 12 epochs of FCN.

In []:

In []: